

Muhammad Mahad Khan

24K-0961

BCS-5C

CN Assignment 01

SECTION: A

DNS QUERY RESOLUTION USING SOCKET PROGRAMMING

Code:

“

```
#!/usr/bin/env python3
```

```
"""
```

DNS Query Client - Computer Networks Assignment #1

Manually builds a DNS query, sends it over UDP port 53, and parses the raw DNS response.

No high-level resolver such as `gethostbyname()` is used.

```
"""
```

```
import socket
```

```
import struct
```

```
import random
```

```
DNS_PORT = 53
```

```
TYPE_A = 1
```

```
CLASS_IN = 1
```

```
TYPE_NAMES = {  
    1: "A", 2: "NS", 5: "CNAME", 6: "SOA", 12: "PTR",  
    15: "MX", 16: "TXT", 28: "AAAA", 33: "SRV", 255: "ANY"  
}
```

```
RCODE_NAMES = {  
    0: "NOERROR",  
    1: "FORMERR",  
    2: "SERVFAIL",  
    3: "NXDOMAIN",  
    4: "NOTIMP",  
    5: "REFUSED"  
}
```

```
def encode_domain_name(domain):  
    """Convert a domain name like www.example.com into DNS wire format."""  
    domain = domain.strip().rstrip(".")  
    if not domain:  
        raise ValueError("Domain name is empty.")  
  
    labels = domain.split(".")  
    encoded = b""  
    total_len = 1 # final zero byte  
  
    for label in labels:  
        if not label:
```

```

        raise ValueError("Domain contains an empty label.")
    if len(label) > 63:
        raise ValueError(f"Label '{label}' exceeds 63 characters.")

    try:
        lb = label.encode("ascii")
    except UnicodeEncodeError:
        raise ValueError("Only ASCII domain names are supported in this implementation.")

    if len(lb) > 63:
        raise ValueError(f"Encoded label '{label}' exceeds 63 bytes.")

    encoded += bytes([len(lb)]) + lb
    total_len += len(lb) + 1

    if total_len > 255:
        raise ValueError("Encoded domain name exceeds 255 bytes.")

    encoded += b"\x00"
    return encoded

def build_dns_query(domain, txid=None, qtype=TYPE_A):
    """Build a standard DNS query message."""
    if txid is None:
        txid = random.randint(0, 0xFFFF)

```

```

flags = 0x0100 # Recursion Desired = 1
header = struct.pack("!HHHHHH", txid, flags, 1, 0, 0, 0)
qname = encode_domain_name(domain)
question = qname + struct.pack("!HH", qtype, CLASS_IN)

return header + question, txid

```

```

def read_name(data, offset):

```

```

    """

```

```

    Read a DNS name from packet data.

```

```

    Supports DNS compression pointers.

```

```

    Returns (name, next_offset).

```

```

    """

```

```

    labels = []

```

```

    next_offset = None

```

```

    seen_offsets = set()

```

```

    while True:

```

```

        if offset >= len(data):

```

```

            raise ValueError("DNS name goes past end of packet.")

```

```

        length = data[offset]

```

```

        # Compression pointer

```

```

        if (length & 0xC0) == 0xC0:

```

```

            if offset + 1 >= len(data):

```

```
raise ValueError("Truncated compression pointer.")
```

```
if next_offset is None:
```

```
    next_offset = offset + 2
```

```
pointer = ((length & 0x3F) << 8) | data[offset + 1]
```

```
if pointer >= len(data):
```

```
    raise ValueError("Compression pointer out of range.")
```

```
if pointer in seen_offsets:
```

```
    raise ValueError("Compression pointer loop detected.")
```

```
seen_offsets.add(pointer)
```

```
offset = pointer
```

```
continue
```

```
if (length & 0xC0) != 0:
```

```
    raise ValueError("Invalid DNS label length.")
```

```
offset += 1
```

```
if length == 0:
```

```
    if next_offset is None:
```

```
        next_offset = offset
```

```
    break
```

```
if offset + length > len(data):
```

```
    raise ValueError("DNS label goes past end of packet.")
```

```
label = data[offset:offset + length].decode("ascii", errors="replace")
```

```
labels.append(label)
```

```
offset += length
```

```
return ".".join(labels), next_offset
```

```
def parse_dns_response(data):
```

```
    """Parse DNS response header, question section, and answer section."""
```

```
    if len(data) < 12:
```

```
        raise ValueError("DNS response shorter than 12-byte header.")
```

```
    txid, flags, qdcount, ancound, nscount, arcount = struct.unpack(
```

```
        "!HHHHHH", data[:12]
```

```
)
```

```
    qr = (flags >> 15) & 1
```

```
    opcode = (flags >> 11) & 0xF
```

```
    aa = (flags >> 10) & 1
```

```
    tc = (flags >> 9) & 1
```

```
    rd = (flags >> 8) & 1
```

```
    ra = (flags >> 7) & 1
```

```
    rcode = flags & 0xF
```

```
offset = 12
```

```
questions = []
```

```
for _ in range(qdcount):
```

```
    name, offset = read_name(data, offset)
```

```
    if offset + 4 > len(data):
```

```
        raise ValueError("Truncated question section.")
```

```
    qtype, qclass = struct.unpack("!HH", data[offset:offset + 4])
```

```
    offset += 4
```

```
    questions.append({
```

```
        "name": name,
```

```
        "type": qtype,
```

```
        "class": qclass
```

```
    })
```

```
answers = []
```

```
for _ in range(ancount):
```

```
    name, offset = read_name(data, offset)
```

```
    if offset + 10 > len(data):
```

```
        raise ValueError("Truncated answer record.")
```

```

rtype, rclass, ttl, rdlength = struct.unpack(
    "!HHIH", data[offset:offset + 10]
)
offset += 10

if offset + rdlength > len(data):
    raise ValueError("Truncated answer RDATA.")

rdata_start = offset
rdata = data[offset:offset + rdlength]
offset += rdlength

data_str = rdata.hex()

if rtype == TYPE_A and rdlength == 4:
    data_str = socket.inet_ntoa(rdata)

elif rtype == 28 and rdlength == 16: # AAAA
    data_str = socket.inet_ntop(socket.AF_INET6, rdata)

elif rtype in (2, 5, 12): # NS, CNAME, PTR
    try:
        data_str, _ = read_name(data, rdata_start)
    except ValueError:
        data_str = rdata.hex()

```

```
elif rtype == 15 and rdlength >= 3: # MX
    try:
        pref = struct.unpack("!H", rdata[:2])[0]
        exch, _ = read_name(data, rdata_start + 2)
        data_str = f"{pref} {exch}"
    except ValueError:
        data_str = rdata.hex()
```

```
answers.append({
    "name": name,
    "type": rtype,
    "class": rclass,
    "ttl": ttl,
    "rdlength": rdlength,
    "data": data_str
})
```

```
return {
    "txid": txid,
    "flags": flags,
    "qr": qr,
    "opcode": opcode,
    "aa": aa,
    "tc": tc,
    "rd": rd,
    "ra": ra,
```

```
"rcode": rcode,  
"qdcoun": qdcoun,  
"ancoun": ancoun,  
"nscoun": nscoun,  
"arcount": arcount,  
"questions": questions,  
"answers": answers  
}
```

```
def resolve(domain, dns_server, timeout=5.0):  
    """Send DNS query and return parsed response."""  
    query, txid = build_dns_query(domain)  
  
    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)  
    sock.settimeout(timeout)  
  
    try:  
        sock.sendto(query, (dns_server, DNS_PORT))  
        response, server_addr = sock.recvfrom(4096)  
    finally:  
        sock.close()  
  
    parsed = parse_dns_response(response)  
  
    if parsed["txid"] != txid:  
        raise ValueError(
```

```
        f"Transaction ID mismatch: sent {txid}, received {parsed['txid']}."
    )
```

```
    return parsed
```

```
def print_response(domain, dns_server, parsed):
```

```
    """Print parsed DNS response."""
```

```
    print("\n" + "=" * 70)
```

```
    print(f"DNS Server    : {dns_server}:{DNS_PORT}")
```

```
    print(f"Query Name     : {domain}")
```

```
    print(f"Query Type    : A")
```

```
    print(f"Transaction ID : {parsed['txid']} (0x{parsed['txid']:04X})")
```

```
    print(
```

```
        f"Flags        : QR={parsed['qr']], Opcode={parsed['opcode']], "
```

```
        f"AA={parsed['aa']], TC={parsed['tc']], RD={parsed['rd']], "
```

```
        f"RA={parsed['ra']], RCODE={parsed['rcode']} "
```

```
        f"({RCODE_NAMES.get(parsed['rcode'], 'UNKNOWN')})"
```

```
    )
```

```
    print(
```

```
        f"Counts        : QD={parsed['qdcount']], AN={parsed['ancount']], "
```

```
        f"NS={parsed['nscount']], AR={parsed['arcount']}"
```

```
    )
```

```
if parsed["questions"]:
```

```
    q = parsed["questions"][0]
```

```
    print(
```

```

        f"Question Section: {q['name']} "
        f"type={TYPE_NAMES.get(q['type'], q['type'])} "
        f"class={q['class']}"
    )

```

```

if parsed["rcode"] != 0:

```

```

    print(f"Response Status : {RCODE_NAMES.get(parsed['rcode'], 'UNKNOWN')}")

```

```

if not parsed["answers"]:

```

```

    print("Answer Section : (none)")

```

```

else:

```

```

    print("Answer Section :")

```

```

    for i, ans in enumerate(parsed["answers"], 1):

```

```

        type_name = TYPE_NAMES.get(ans["type"], str(ans["type"]))

```

```

        print(

```

```

            f" [{i}] Name={ans['name']} "

```

```

            f"Type={type_name} Class={ans['class']} "

```

```

            f"TTL={ans['ttl']} Data={ans['data']}"

```

```

        )

```

```

a_records = [

```

```

    a for a in parsed["answers"]

```

```

    if a["type"] == TYPE_A and a["class"] == CLASS_IN

```

```

]

```

```

if a_records:

```

```

        print("Resolved IPv4 Address(es):")

        for a in a_records:

            print(f" {a['data']} (TTL={a['ttl']} seconds)")

        else:

            print("Resolved IPv4 Address: (no A record found)")


def main():

    print("DNS Query Client (UDP, port 53)")

    dns_server = input("Enter DNS server IP address [default 8.8.8.8]: ").strip()

    if not dns_server:

        dns_server = "8.8.8.8"

    while True:

        domain = input("\nEnter domain name (or blank/quit to exit): ").strip()

        if domain.lower() in ("", "quit", "exit", "q"):

            print("Exiting.")

            break

        try:

            parsed = resolve(domain, dns_server)

            print_response(domain, dns_server, parsed)

        except socket.timeout:

            print(f"Error: Request timed out while waiting for DNS server {dns_server}.")

```

```

except ValueError as e:

    print(f"Error: {e}")

except OSError as e:

    print(f"Network/socket error: {e}")

except Exception as e:

    print(f"Unexpected error: {e}")

```

```

if __name__ == "__main__":

    main()

```

“

Output:

```

PS C:\Users\Admin\OneDrive\Desktop\python> python .\dns_client.py
DNS Query Client (UDP, port 53)
Enter DNS server IP address [default 8.8.8.8]: 8.8.8.8

Enter domain name (or blank/quit to exit): www.youtube.com

=====
DNS Server      : 8.8.8.8:53
Query Name     : www.youtube.com
Query Type     : A
Transaction ID  : 44522 (0xADEA)
Flags          : QR=1, Opcode=0, AA=0, TC=0, RD=1, RA=1, RCODE=0 (NOERROR)
Counts         : QD=1, AN=8, NS=0, AR=0
Question Section: www.youtube.com type=A class=1
Answer Section :
[1] Name=www.youtube.com Type=A Class=1 TTL=36 Data=142.251.155.4
[2] Name=www.youtube.com Type=A Class=1 TTL=36 Data=142.251.150.4
[3] Name=www.youtube.com Type=A Class=1 TTL=36 Data=142.251.156.4
[4] Name=www.youtube.com Type=A Class=1 TTL=36 Data=142.251.157.4
[5] Name=www.youtube.com Type=A Class=1 TTL=36 Data=142.251.153.4
[6] Name=www.youtube.com Type=A Class=1 TTL=36 Data=142.251.152.4
[7] Name=www.youtube.com Type=A Class=1 TTL=36 Data=142.251.154.4
[8] Name=www.youtube.com Type=A Class=1 TTL=36 Data=142.251.151.4
Resolved IPv4 Address(es):
142.251.155.4 (TTL=36 seconds)
142.251.150.4 (TTL=36 seconds)
142.251.156.4 (TTL=36 seconds)
142.251.157.4 (TTL=36 seconds)
142.251.153.4 (TTL=36 seconds)
142.251.152.4 (TTL=36 seconds)
142.251.154.4 (TTL=36 seconds)
142.251.151.4 (TTL=36 seconds)

Enter domain name (or blank/quit to exit): www.google.com

=====
DNS Server      : 8.8.8.8:53
Query Name     : www.google.com
Query Type     : A

```

```
File Edit Selection View Go Run Terminal Help python
Problems Output Debug Console Terminal Ports
PS C:\Users\Admin\OneDrive\Desktop\python> python .\dns_client.py

Enter domain name (or blank/quit to exit): www.google.com

=====
DNS Server      : 8.8.8.8:53
Query Name      : www.google.com
Query Type      : A
Transaction ID   : 43081 (0xA849)
Flags           : QR=1, Opcode=0, AA=0, TC=0, RD=1, RA=1, RCODE=0 (NOERROR)
Counts          : QD=1, AN=8, NS=0, AR=0
Question Section: www.google.com type=A class=1
Answer Section  :
[1] Name=www.google.com Type=A Class=1 TTL=206 Data=142.251.157.119
[2] Name=www.google.com Type=A Class=1 TTL=206 Data=142.251.151.119
[3] Name=www.google.com Type=A Class=1 TTL=206 Data=142.251.154.119
[4] Name=www.google.com Type=A Class=1 TTL=206 Data=142.251.153.119
[5] Name=www.google.com Type=A Class=1 TTL=206 Data=142.251.155.119
[6] Name=www.google.com Type=A Class=1 TTL=206 Data=142.251.152.119
[7] Name=www.google.com Type=A Class=1 TTL=206 Data=142.251.156.119
[8] Name=www.google.com Type=A Class=1 TTL=206 Data=142.251.150.119
Resolved IPv4 Address(es):
142.251.157.119 (TTL=206 seconds)
142.251.151.119 (TTL=206 seconds)
142.251.154.119 (TTL=206 seconds)
142.251.153.119 (TTL=206 seconds)
142.251.155.119 (TTL=206 seconds)
142.251.152.119 (TTL=206 seconds)
142.251.156.119 (TTL=206 seconds)
142.251.150.119 (TTL=206 seconds)

Enter domain name (or blank/quit to exit): www.wikipedia.org

=====
DNS Server      : 8.8.8.8:53
Query Name      : www.wikipedia.org
Query Type      : A
Transaction ID   : 56182 (0xB76)
Flags           : QR=1, Opcode=0, AA=0, TC=0, RD=1, RA=1, RCODE=0 (NOERROR)

Ln 15, Col 11 Spaces: 4 UTF-8 CRLF Python Python 3.14.4 Go Live
12:10 AM 9/29/2026
```

```
File Edit Selection View Go Run Terminal Help python
Problems Output Debug Console Terminal Ports powershell
PS C:\Users\Admin\OneDrive\Desktop\python> python .\dns_client.py
Enter domain name (or blank/quit to exit): www.wikipedia.org

=====
DNS Server      : 8.8.8.8:53
Query Name      : www.wikipedia.org
Query Type      : A
Transaction ID   : 56182 (0x0B76)
Flags           : QR=1, Opcode=0, AA=0, TC=0, RD=1, RA=1, RCODE=0 (NOERROR)
Counts          : QD=1, AN=2, NS=0, AR=0
Question Section: www.wikipedia.org type=A class=1
Answer Section  :
[1] Name=www.wikipedia.org Type=CNAME Class=1 TTL=28914 Data=dyna.wikimedia.org
[2] Name=dyna.wikimedia.org Type=A Class=1 TTL=63 Data=103.102.166.224
Resolved IPv4 Address(es):
103.102.166.224 (TTL=63 seconds)

Enter domain name (or blank/quit to exit): www.mit.edu

=====
DNS Server      : 8.8.8.8:53
Query Name      : www.mit.edu
Query Type      : A
Transaction ID   : 39237 (0x9945)
Flags           : QR=1, Opcode=0, AA=0, TC=0, RD=1, RA=1, RCODE=0 (NOERROR)
Counts          : QD=1, AN=3, NS=0, AR=0
Question Section: www.mit.edu type=A class=1
Answer Section  :
[1] Name=www.mit.edu Type=CNAME Class=1 TTL=861 Data=www.mit.edu.edgekey.net
[2] Name=www.mit.edu.edgekey.net Type=CNAME Class=1 TTL=60 Data=e9566.dscb.akamaiedge.net
[3] Name=e9566.dscb.akamaiedge.net Type=A Class=1 TTL=20 Data=173.222.144.77
Resolved IPv4 Address(es):
173.222.144.77 (TTL=20 seconds)

Enter domain name (or blank/quit to exit): www.nu.edu.pk

=====
DNS Server      : 8.8.8.8:53
```

```

File Edit Selection View Go Run Terminal Help
PS C:\Users\Admin\OneDrive\Desktop\python> python .\dns_client.py
Enter domain name (or blank/quit to exit): www.nu.edu.pk

=====
DNS Server      : 8.8.8.8:53
Query Name     : www.nu.edu.pk
Query Type    : A
Transaction ID  : 44880 (0xAFE0)
Flags          : QR=1, Opcode=0, AA=0, TC=0, RD=1, RA=1, RCODE=0 (NOERROR)
Counts        : QD=1, AN=2, NS=0, AR=0
Question Section: www.nu.edu.pk type=A class=1
Answer Section :
[1] Name=www.nu.edu.pk type=NAME Class=1 TTL=13178 Data=nu.edu.pk
[2] Name=nu.edu.pk type=A Class=1 TTL=13178 Data=203.124.44.78
Resolved IPv4 Address(es):
203.124.44.78 (TTL=13178 seconds)

Enter domain name (or blank/quit to exit): quit
Exiting.
PS C:\Users\Admin\OneDrive\Desktop\python> python .\dns_client.py
DNS Query Client (UDP, port 53)
Enter DNS server IP address [default 8.8.8.8]: 1.1.1.1

Enter domain name (or blank/quit to exit): cdn.streamhub.com

=====
DNS Server      : 1.1.1.1:53
Query Name     : cdn.streamhub.com
Query Type    : A
Transaction ID  : 30633 (0x77A9)
Flags          : QR=1, Opcode=0, AA=0, TC=0, RD=1, RA=1, RCODE=3 (NOERROR)
Counts        : QD=1, AN=0, NS=1, AR=0
Question Section: cdn.streamhub.com type=A class=1
Response Status : NOERROR
Answer Section : (none)

Enter domain name (or blank/quit to exit): www.netlearn.edu

```

```

File Edit Selection View Go Run Terminal Help
Problems Output Debug Console Terminal Ports
PS C:\Users\Admin\OneDrive\Desktop\python> python .\dns_client.py
Resolved IPv4 Address(es):
203.124.44.78 (TTL=13178 seconds)

Enter domain name (or blank/quit to exit): quit
Exiting.
PS C:\Users\Admin\OneDrive\Desktop\python> python .\dns_client.py
DNS Query client (UDP, port 53)
Enter DNS server IP address [default 8.8.8.8]: 1.1.1.1

Enter domain name (or blank/quit to exit): cdn.streamhub.com

=====
DNS Server      : 1.1.1.1:53
Query Name      : cdn.streamhub.com
Query Type      : A
Transaction ID   : 30633 (0x77AB)
Flags           : QR=1, Opcode=0, AA=0, TC=0, RD=1, RA=1, RCODE=3 (NXDOMAIN)
Counts          : QD=1, AN=0, NS=1, AR=0
Question Section: cdn.streamhub.com type=A class=1
Response Status : NXDOMAIN
Answer Section  : (none)

Enter domain name (or blank/quit to exit): www.netlearn.edu

=====
DNS Server      : 1.1.1.1:53
Query Name      : www.netlearn.edu
Query Type      : A
Transaction ID   : 54504 (0xD4E8)
Flags           : QR=1, Opcode=0, AA=0, TC=0, RD=1, RA=1, RCODE=3 (NXDOMAIN)
Counts          : QD=1, AN=0, NS=1, AR=0
Question Section: www.netlearn.edu type=A class=1
Response Status : NXDOMAIN
Answer Section  : (none)

Enter domain name (or blank/quit to exit): quit
Exiting.
PS C:\Users\Admin\OneDrive\Desktop\python>

```

SECTION B:
NUMERICALS

24K-0961

BCS-5C

M. Mahad Khan

Dated: 23-09-26

CN

Assignment : 01

Section - B

Numericals

Q:1) DNS Resolution time

Give iterative queries for www.netlearn.edu:

- Root DNS server = 60ms
- TLD DNS server = 50ms
- Authoritative DNS server = 40ms

a) Total DNS resolution time:

The Local DNS must query:

Root → TLD → Authoritative

Total time:

$$60 + 50 + 40 = 150 \text{ ms}$$

b) If Root and TLD are already cached

The local DNS server only queries the authoritative server:

Time:

$$40 \text{ ms}; \text{ Time saved} = 150 - 40 = 110 \text{ ms}$$

110 ms saved.

Ans

Dated: _____

Q:2) DNS Caching and TTL

Record for `cdn.streamhub.com`:

- Cached at 10:00:00 AM
- TTL = 300 seconds = 5 minutes.
- Valid until 10:05:00 AM
- Full resolution = 150 ms
- Cache lookup = 2 ms

a) Query at 10:03:00 AM:-

Elapsed time:

$$10:03:00 - 10:00:00 = 3\text{min} = 180\text{ sec}$$

Since, $180 < 300$

The cached entry is still valid.

Time saved by using cache:

$$150 - 2 = 148\text{ ms; saved.}$$

b) Query at 10:07:00 AM.

$$10:07:00 - 10:00:00 = 7\text{mins} = 420\text{ sec}$$

Since, $420 > 300$

The TTL has expired; The cached entry is not valid.
A Full resolution is required.

Ans:

Dated: _____

Q:3) Transmission Delay:

File size: $12\text{MB} = 12 \times 8 = 96\text{ Mega bits}$

link 1 speed = 20Mbps

$$T_1 = \frac{96}{20} = 4.8\text{s}$$

link 2 speed = 100Mbps

$$T_2 = \frac{96}{100} = 0.96\text{s}$$

Total transmission Delay:

$$T_{\text{total}} = 4.8 + 0.96 = 5.76\text{s} \quad \text{Ans:}$$

Q:4) End to End Delay:

Packet size = 8000 bits

L1:

speed = $40\text{Mbps} = 40 \times 10^6\text{ bps}$

T_{Delay} :

$$T_{D1} = \frac{8000}{40,000,000} = 0.0002\text{s} = 0.2\text{ms}$$

$$P_{\text{Delay}} = P_{D1} = 20\text{ms}$$

$$T_1 = T_{D1} + P_{D1} = 20 + 0.2 = 20.2\text{ms}$$

Dated: _____

L2:

$$\text{Speed} = 80 \text{ Mbps} = 80 \times 10^6 \text{ bps}$$

$$T_{D2} = \frac{8000}{80,000,000} = 0.0001 \text{ s} = 0.1 \text{ ms}$$

$$P_{D2} = 30 \text{ ms}$$

$$T_2 = T_{D2} + P_{D2} = 0.1 + 30 = 30.1 \text{ ms}$$

L3:

$$\text{Speed} = 500 \text{ Mbps} = 500 \times 10^6$$

$$T_{D3} = \frac{8000}{500,000,000} = 0.000016 \text{ s} = 0.016 \text{ ms}$$

$$P_{D3} = 5 \text{ ms}$$

$$T_3 = T_{D3} + P_{D3} = 0.016 + 5 = 5.016 \text{ ms}$$

Total end to end Delay

$$T_{\text{total}} = T_1 + T_2 + T_3 = 20.2 + 30.1 + 5.016 \\ = 55.316 \text{ ms}$$

$$T_{\text{total}} = 55.316 \text{ ms}$$

Ans.

Dated: _____

Q.5) Web Cache / Hit Ratio:

Given:

- Request = 5000 per hour
- Avg object size = 400 Kb
- Cache Hit ratio = 75%

a) Total traffic per hour without caching

$$5000 \times 400 = 2,000,000 \text{ Kb/hr}$$

$$2,000,000 \text{ Kb/hr} = 2,000 \text{ Mb/hr} = 250 \text{ MB/hr}$$

b) Traffic per hr reaching origin server after caching:

Miss ratio:

$$100\% - 75\% = 25\%$$

Requests reaching origin:

$$0.25 \times 5000 = 1250 \text{ requests per hr}$$

Traffic =

$$1250 \times 400 = 500,000 \text{ Kb/hr}$$

$$500,000 \text{ Kb/hr} = 500 \text{ Mb/hr} = 62.5 \text{ MB/hr}$$

Dated: 23-09-26

c) Delta traffic saved per hour due to caching

$$2,000,000 - 500,000 = 1,500,000 \text{ kb/hour}$$

$$1,500,000 \text{ kb/hr} = 1,500 \text{ Mb/hr} = 187.5 \text{ MB/hr}$$

Ans:

THE END

THE END!!